```
#include "Task.h"
#include <iostream>
#include <sstream>
using namespace std;

// Constructor - calls PDAItem constructor first, then stores Task's own attributes
Task::Task(int id, const string& title, const string& dueDate, const string& priority, bool
completed)
    : PDAItem(id, title) {
    this->dueDate = dueDate;
    this->priority = priority;
    this->completed = completed;
}

// Returns the due date
string Task::getDueDate() const {
    return dueDate;
}

// Returns the priority
string Task::getPriority() const {
    return priority;
}

// Returns true if task is completed, false if not
bool Task::isCompleted() const {
    return completed;
}

// Sets due date - rejects empty string and wrong format
void Task::setDueDate(const string& dueDate) {
    if (dueDate.empty()) {
        cout << "Error: due date cannot be empty." << endl;
        return;
    }
    // Check format YYYY-MM-DD
    if (dueDate.length() != 10 || dueDate[4] != '-' || dueDate[7] != '-') {
        cout << "Error: due date must be in YYYY-MM-DD format." << endl;
        return;
    }
    this->dueDate = dueDate;
}

// Sets priority - only accepts High, Medium, or Low
void Task::setPriority(const string& priority) {
    if (priority != "High" && priority != "Medium" && priority != "Low") {
        cout << "Error: priority must be High, Medium, or Low." << endl;
```

```
        return;
    }
    this->priority = priority;
}

// Sets completed status
void Task::setCompleted(bool status) {
    this->completed = status;
}

// Marks task as done
void Task::markComplete() {
    this->completed = true;
}

// Displays all task details in the terminal
void Task::display() const {
    cout << "ID: " << getId() << endl;
    cout << "Title: " << getTitle() << endl;
    cout << "Due Date: " << dueDate << endl;
    cout << "Priority: " << priority << endl;
    cout << "Status: " << (completed ? "Done" : "Pending") << endl;
}

// Saves task to file in format: ID|Title|DueDate|Priority|0or1
void Task::saveToFile(ofstream& file) const {
    file << getId() << "|"
         << getTitle() << "|"
         << dueDate << "|"
         << priority << "|"
         << (completed ? 1 : 0)
         << endl;
}

// Reads task from file in format: ID|Title|DueDate|Priority|0or1
void Task::loadFromFile(ifstream& file) {
    string line;
    if (getline(file, line)) {
        stringstream ss(line);
        string idStr, title, dueDate, priority, completedStr;

        getline(ss, idStr, '|');
        getline(ss, title, '|');
        getline(ss, dueDate, '|');
        getline(ss, priority, '|');
        getline(ss, completedStr, '|');
```

```
        setId(stoi(idStr));
        setTitle(title);
        this->dueDate = dueDate;
        this->priority = priority;
        this->completed = (completedStr == "1");
    }
}
```